Evaluating LLM Pipelines

Metrics, Rubrics, and LLM-as-a-Judge

Dr. Papa-Séga WADE

AIMS Senegal | Module 2 - Session 8

MODULE 2 - Session 8: evaluation, rubrics, and LLM-as-a-Judge

"Measure before you trust"

Where We Stopped in Session 7

Recap

You can now **prompt**, **retrieve**, and **adapt**. Three levers, three trade-offs. The next question is the only one that really matters in production: **does any of this actually work?**

Prompting → **RAG** → **Fine-tuning** → **Evaluation (today)**

We are here

Baseline Objective

By the end of today's session

You will know how to **rigorously evaluate** the outputs of an LLM pipeline using classic metrics, LLM-as-a-Judge, rubrics, and RAGAS.

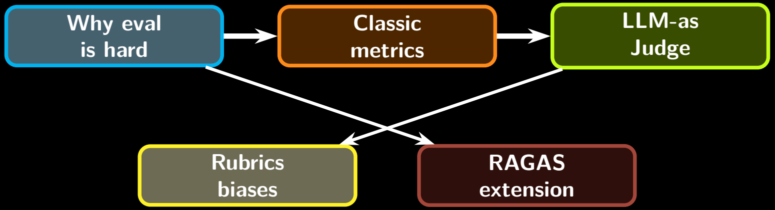
You will be able to

- explain why LLM eval is harder than classical ML eval,
- pick a metric that fits your task,
- run an LLM-as-a-Judge on your S6 mini-RAG,
- interpret evaluation results and identify failure cases.

Guiding question

For the AIMS-Senegal assistant you will build, what is the smallest set of metrics that tells you it is **good enough** to ship to a real student?

Session Roadmap



Core message

Without evaluation, "it works on my prompt" is not engineering. With evaluation, you can ship.

Why LLM Evaluation Is Hard

Classical ML

For a classifier, accuracy is a number. The label is either right or wrong. Easy to aggregate, easy to compare.

LLM outputs are different

- answers are open-ended text,
- many correct answers exist,
- style and tone matter,
- factual correctness depends on context.

AIMS example

Q: `"Quel est le but du module GAAI?"`

Acceptable answers:

- "Apprendre a deployer des LLM en Afrique."
- "Former des ingenieurs IA appliquee."
- "Maitriser prompting, RAG et fine-tuning."

All three are valid. Exact-match scores 0.

Three Dimensions of LLM Quality

Correctness (facts right) → **Groundedness (uses context)** → **Style (format, tone)**

What this means

A good evaluation framework measures **at least these three**, separately. A model can be factually right but stylistically off, or stylistically perfect but hallucinated.

Concrete instance

A Wolof health assistant might answer in correct French (style fail), or in fluent Wolof but with the wrong dosage (correctness fail), or quote an unrelated section of the protocol (groundedness fail).

Classic Metrics: Exact Match and F1

Exact Match (EM)

1 if prediction equals reference, 0 otherwise.

When it works

Tasks with a unique short answer: dates, numbers, named entities, structured extraction.

When it fails

Open answers. `"Dakar"` vs `"la ville de Dakar"` scores 0.

Token-level F1

Treat answer as a bag of tokens. Compute precision, recall, F1 against the reference.

Useful for

QA over short factual answers (SQuAD-style). More tolerant than EM.

Limit

Still surface-form. `"poisson"` vs `"thiof"` scores 0.

Classic Metrics: BLEU and ROUGE

BLEU

N-gram **precision** of prediction against one or more references. Originally designed for machine translation.

Use for

Translation tasks (e.g. English $\to$ Wolof), where token-level overlap is meaningful.

ROUGE

N-gram and longest-common-subsequence **recall**. Standard for summarization.

Use for

Summarizing AIMS lecture notes, ANSD reports, scientific abstracts.

Shared limit

Both reward surface overlap, not meaning. A correct paraphrase scores low. Do not use them as the **only** metric.

Classic Metrics: Perplexity

Definition

For a sequence $w_1 \dots w_T$:

$$\mathrm{PPL} = \exp\left(-\frac{1}{T} \sum_t \log p(w_t \mid w_{<t}) \right)$$

Lower is better. Measures how "surprised" the model is by held-out text.

Use it for

Comparing two language models on the same corpus. Quick sanity check after fine-tuning.

Do not use it for

- ranking models on a downstream task,
- evaluating instruction-following,
- comparing models with **different tokenizers** (numbers are not comparable).

Heuristic

Perplexity is a thermometer. Useful, but does not tell you the meal is good.

Limits of Classic Metrics, Summarized

Metric	Best for	Main weakness
Exact Match	short factual answers	any rephrasing scores 0
Token F1	SQuAD-style short QA	ignores meaning, only surface
BLEU	translation	rewards copy, punishes paraphrase
ROUGE	summarization	surface n-gram overlap
Perplexity	LM-vs-LM on same corpus	not comparable across tokenizers

Take-away

Use these metrics to detect **regressions**, not to claim absolute quality.

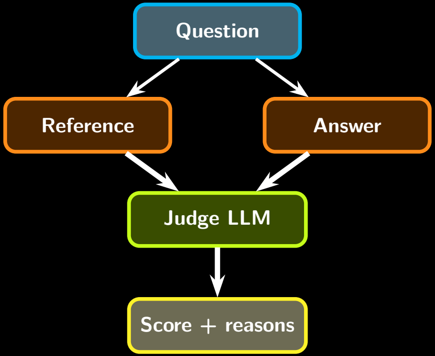
LLM-as-a-Judge: The Idea

Principle

Use a strong LLM (GPT-4, Claude, Llama 3 70B) to **score** another LLM's output against a reference, a rubric, or a context.

Why we use it

- scales to thousands of examples,
- handles paraphrase and meaning,
- lets us score open-ended answers,
- cheaper than human labeling.



LLM-as-a-Judge: Known Biases

Position bias

Judges tend to prefer the **first** answer in a pairwise comparison. Always randomize order.

Verbosity bias

Judges over-reward longer, more elaborate answers, even when the short answer is correct.

Self-preference bias

A model judging outputs from the same family rates them higher. Use a different judge model than the one being scored.

Mitigations that work

- **shuffle order** in pairwise mode,
- **use a rubric**, not a free score,
- **cap output length** on judged model and judge,
- use **majority vote** of 3 judges,
- **calibrate** with 20 human-labeled samples.

A Simple Rubric for the AIMS RAG

Score on a 1-5 scale, four axes

1. **Correctness**: is the answer factually right given the AIMS document?
2. **Groundedness**: are claims supported by the retrieved chunks?
3. **Refusal calibration**: does the model say "I don't know" when it should?
4. **Style**: is the language register appropriate (academic French)?

Composite score

Compute the mean per axis on a 20-question evaluation set. A pipeline that scores ≥ 4.0 on Correctness AND ≥ 4.0 on Groundedness is "ship-ready" for an internal AIMS pilot.

Short Lab: Evaluate the S6 Mini-RAG

What you will do (30 min)

Reuse your Session 6 RAG over the AIMS handbook. Prepare 10 evaluation questions, run the pipeline, then judge each answer with an LLM judge.

Steps

1. Write 10 questions with reference answers (from the handbook).
2. Run RAG, collect predictions.
3. Implement an LLM-as-a-Judge with the 4-axis rubric.
4. Aggregate scores, identify failure cases.

Suggested questions

- "Quel est le seuil de validation d'un module?"
- "Combien de credits ECTS au S2?"
- "Quelle est la procedure d'absence justifiee?"
- "Quelle annee scolaire couvre ce reglement?"
- ... (mix easy / hard / out-of-corpus)

Lab Code: A Minimal LLM-as-a-Judge

```python
from transformers import pipeline
judge = pipeline("text-generation", model="meta-llama/Llama-3.1-8B-Instruct")

JUDGE_PROMPT = """You are an evaluator. Score the answer on 1-5 for each axis.
Return ONLY a JSON: {"correctness": int, "groundedness": int, "refusal": int, "style": int, "reason": str}.

Question: {q}
Reference: {ref}
Context used by the model: {ctx}
Model answer: {ans}
"""

def score(q, ref, ctx, ans):
    p = JUDGE_PROMPT.format(q=q, ref=ref, ctx=ctx, ans=ans)
    out = judge(p, max_new_tokens=200, do_sample=False)[0]["generated_text"]
    return out  # parse JSON, append to results

eval_set = [
    {"q": "Quel est le seuil de validation d'un module?", "ref": "..."},
    # ... 10 items
]
# loop, score, average per axis
```

Reading the Eval Output

What good looks like

- correctness ≥ 4 on average,
- groundedness ≥ 4,
- refusal: 5 on out-of-corpus questions,
- style: 4 or 5 across the board.

Common failure patterns

- high correctness, low groundedness $\rightarrow$ the model knew the fact, ignored your context,
- low refusal $\rightarrow$ hallucinates outside the corpus,
- low style $\rightarrow$ tone wrong, often a fine-tune fix.

Action item

For every failure case, write one sentence: **which lever fixes it**? Prompt change, retrieval change, fine-tune, or data fix?

Advanced Extension: RAGAS

What RAGAS gives you

An open-source framework that automates LLM-as-a-Judge for RAG pipelines, with named metrics:

- **faithfulness**: claims supported by context,
- **answer relevancy**: answer matches the question,
- **context precision**: retrieved chunks are actually used,
- **context recall**: needed chunks were retrieved.

Why it matters

Standardizes the vocabulary across teams, makes evaluations **reproducible**, plays well with LangChain and LlamaIndex.

For advanced profiles

Run RAGAS on your S6 mini-RAG and compare its metrics to your hand-built rubric.

RAGAS in 15 Lines

```python
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision, context_recall
from datasets import Dataset

samples = [
    {
        "question": "Quel est le seuil de validation d'un module?",
        "answer": "10 sur 20 selon le reglement AIMS.",
        "contexts": ["...passage extrait du PDF AIMS..."],
        "ground_truth": "Le seuil est de 10 sur 20.",
    },
    # ... 20 such items
]

ds = Dataset.from_list(samples)
report = evaluate(ds, metrics=[faithfulness, answer_relevancy, context_precision, context_recall])
print(report)
```

What to Remember

1. LLM eval is harder than classical ML eval: many right answers, style and grounding matter.
2. Classic metrics (EM, F1, BLEU, ROUGE, perplexity) detect regressions but do not measure quality.
3. LLM-as-a-Judge scales human evaluation, but bring **rubrics, randomization, and a different judge**.
4. Always evaluate three dimensions: **correctness**, **groundedness**, **style**.
5. Evaluation must include refusal behavior and failure-case analysis.
6. RAGAS is a good standardization layer for RAG pipelines.

Key Resources

Evaluation

- RAGAS framework (GitHub + docs)
- LangChain Evaluation toolkit
- Hugging Face `evaluate` library
- MT-Bench, AlpacaEval (LLM-as-Judge canonical setups)

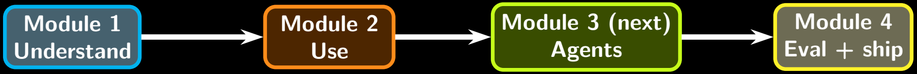
RAG evaluation

- RAGAS named metrics
- Faithfulness and answer relevancy
- Context precision and context recall
- Manual rubrics for calibration

Bridge to Module 3: Agents

What comes next

Module 3 turns the LLM from a chatbot into an **agent**: it uses tools, holds memory, and plans multi-step actions. You will use everything from Module 2 inside agentic workflows.



Questions?

Measure before you trust. Score with a rubric. Improve from failure cases.

Dr. Papa-Séga WADE

AIMS Senegal | GAAI-AIMS